ASUR

Engineering Handbook

THE CONCEPTS BEHIND ASUR, FOR ENGINEERING INTERVIEWS

This handbook explains every engineering concept used in ASUR. Each concept has up to four parts:

Part	What it gives you
Simple meaning	What the idea is, in plain words.
In ASUR	Where the code uses it, with the file name.
Try it	A few lines you can run in a terminal to see the idea work.
Q and A	The question an interviewer asks, and a short answer you can say.

Read one chapter at a time. Type every 'Try it' snippet yourself; do not just read it. Then say each answer aloud in your own words. Chapter 14 has coding tasks and the follow-up questions interviewers ask about your own project.

Contents

Where these questions are asked 2

1. Explaining the project 4

2. Hashing and data integrity 5

3. Safe storage on disk 7

4. State machine and gates 9

5. Architecture and design patterns 12

6. The firewall: keeping maker and checker apart 15

7. Determinism and learning 17

8. Video and audio concepts 18

9. AI model concepts 20

10. Testing and code quality 21

11. Security concepts 23

12. Python features used 24

13. One-line revision 25

14. Practice tasks and follow-up questions 26

Where these questions are asked

These questions are asked for backend, Python, platform and applied AI roles, mostly for people with two to six years of experience. They come up in these rounds:

Round	What they ask	Chapters
Project round	'Explain your project.' Then they pick one part and go deeper until you cannot answer. Every company has this round.	1, 14
Fundamentals round	Hashing, race conditions, atomic writes, locks, idempotency.	2, 3
Low-level design or machine coding round	'Design a workflow engine.' 'Design a plugin system.' You write working code in 45 to 90 minutes.	4, 5, 14
System design round	Pipelines, orchestration, review before release, and 'how would you scale this?'	4, 6, 14
Python round	Dataclasses, imports, subprocess, the standard library.	12
Testing and DevOps questions	Test types, flaky tests, CI, linting.	10
Security questions	Injection, trust boundaries, hashing versus signing.	2.4, 11
Domain round at video or AI companies	Codecs, FFmpeg, diffusion, speech models.	8, 9

Role	Most useful chapters
Backend engineer	1 to 6, 10, 11, 14
Python developer	2 to 5, 10, 12, 14
Platform or DevOps engineer	3, 4, 10, 11
Applied AI or ML engineer	5, 6, 7, 9
Video or media engineer	8, 9
Test automation engineer (SDET)	7, 10

What this handbook does not cover

ASUR has no database, no web server and no network code. So you must prepare these from other sources:

Topic	What is asked
Data structures and algorithms	Arrays, trees, graphs, dynamic programming. Most product companies have one or two rounds of this.
SQL and databases	Joins, indexes, transactions, normalisation.
Web and APIs	HTTP, REST, authentication, caching, load balancing.

How to answer a project question

Use the same four steps for every answer about your project.

Step	Say	Example
1. Problem	One line: what needed solving.	'Saved outputs must never be lost or changed silently.'
2. What I built	The mechanism, in two lines.	'Every save writes a new version file with a checksum, created with O_EXCL.'
3. Why this way	The trade-off you accepted.	'It uses more disk, but I get full history and no lost updates.'
4. What I would improve	One honest weakness.	'A crash mid-write can leave a half file. I would write to a temp file and rename.'

1. Explaining the project

Asked in. The project round. Every company, every level.

ASUR turns a one-line idea into a finished short video. It runs fully on your own computer. It needs no login, no API key and no internet.

```
idea -> research -> audience -> strategy -> hooks -> script -> quality gate
     -> visual plan -> small sample (a human approves)
     -> images, voice, music -> edit -> render with FFmpeg
     -> viral check -> publish package (a human approves)
     -> real metrics typed in -> learning for the next video
```

Part	Code	Job
SCRIPT	`asur/script/`	Turns the idea into hooks and a script.
GENERATION	`asur/generation/`, `asur/editing/`	Plans the visuals, makes the assets, edits and renders the video.
VIRAL CHECK	`asur/viral_check/`	Scores the result on 12 points and raises risk flags.
ORCHESTRATOR	`asur/orchestrator/`	Moves the project from one state to the next.
LEARNING	`asur/learning/`	Stores real results so the next video starts smarter.
CORE	`asur/core/`	Hashing, storage, gates and identity. Everything else uses it.

ASUR has eight fixed rules. Each rule comes from one engineering concept, and each concept is explained later in this handbook.

Rule	Simple meaning	Concept
LOCAL-01	Runs only on the local machine.	Local-first design
HONEST-01	Never shows one 'viral %' number.	Honest measurement
FIREWALL-01	The maker and the checker never talk.	Information hiding
GATE-01	If a check is missing, stop.	Fail closed
HUMAN-01	A human approves important steps.	Human in the loop
PROV-01	Every asset records where it came from.	Provenance
EXPLAIN-01	Every decision has a written reason.	Explainability
VERSION-01	Nothing is overwritten.	Append-only storage

Try it. Run this from the ASUR folder.

```
pip install -e .
mkdir demo
asur script --idea "why most people waste money on AI tools" --project demo
ls demo/.script/artifacts           # one folder per stage, one file per version
asur script --idea "why most people waste money on AI tools" --project demo
ls demo/.script/artifacts/script    # v0001 and v0002: nothing was overwritten
```

Q. Tell me about your project.
A. ASUR is a local tool that takes one line of idea and produces a finished short video. The work moves through fixed states, and a gate checks every step. Every output is saved as a versioned file with a checksum, so nothing is lost or changed silently. A human approves the two costly steps: full generation and publishing.

Q. Does an AI model write the script?
A. No. The script stages use fixed templates filled from the idea text. So the same idea always gives the same output, which makes it easy to test. AI models are used only for images, voice and music, and they are optional.

Q. Why local-first and not a cloud service?
A. Three reasons: the user's content stays private, there is no server cost, and there are no keys that can leak. The trade-off is that the user needs a capable machine, and sharing work with a team is harder.

2. Hashing and data integrity

Asked in. The fundamentals round and security questions.

2.1 Hash function (SHA-256)

Simple meaning. A hash function takes any data and gives a short fixed-size fingerprint. The same data always gives the same fingerprint. If even one character changes, the fingerprint changes completely. You cannot get the data back from the fingerprint.

In ASUR. Every saved output carries the SHA-256 fingerprint of its content. The code is in `asur/core/canonical.py`.

Try it. Run this yourself.

```
import hashlib
print(hashlib.sha256(b"hello").hexdigest())
print(hashlib.sha256(b"hellp").hexdigest())    # one letter changed, whole hash changed
```

Q. What are the properties of a good hash function?
A. It is deterministic (same input, same output). It is one-way (you cannot reverse it). A small change in input gives a very different output. And it is very hard to find two inputs with the same hash, which is called a collision.

Q. Why SHA-256 and not MD5?
A. MD5 is broken: people can create two different files with the same MD5 hash. SHA-256 has no known practical collision.

2.2 Canonical JSON

Simple meaning. The same data can be written as JSON in many ways: different key order, different spacing. A hash works on bytes, so two equal objects could give two different hashes. Canonical form means one fixed way of writing the data, so equal data always gives equal bytes.

In ASUR. ASUR sorts the keys, removes extra spaces, uses UTF-8 and adds one newline at the end. Then it hashes those bytes.

```
# asur/core/canonical.py
text = json.dumps(obj, ensure_ascii=False, sort_keys=True,
                  separators=(",", ":"), allow_nan=False) + "\n"
digest = hashlib.sha256(text.encode("utf-8")).hexdigest()
```

Try it. Run this yourself.

```
import json
a = {"x": 1, "y": 2}
b = {"y": 2, "x": 1}
print(a == b)                                                    # True
print(json.dumps(a) == json.dumps(b))                            # False
print(json.dumps(a, sort_keys=True) == json.dumps(b, sort_keys=True))   # True
```

> **Q. Two dictionaries are equal but their hashes differ. Why?**
> **A.** Because the hash is taken over bytes, not over the object. Key order or spacing was different when they were turned into text. The fix is to serialise in one canonical form before hashing.

2.3 Checksum

Simple meaning. A checksum is a hash stored next to the data. Later you hash the data again and compare. If the two values differ, the data was damaged or changed.

In ASUR. `save_artifact()` refuses to save if the checksum does not match the body. `load_latest()` checks it again on every read. Both are in `asur/core/workspace.py`.

Try it. In a terminal, on any file.

```
shasum -a 256 demo/.script/index.json     # note the hash
echo " " >> demo/.script/index.json       # change one byte
shasum -a 256 demo/.script/index.json     # a different hash
```

> **Q. Why verify on read as well as on write?**
> **A.** A file can be damaged after it is written: a disk error, a half-finished write, or someone editing it by hand. Checking on read means bad data is caught before it is used.

2.4 Hashing versus signing

Simple meaning. A hash proves the data did not change. It does not prove who wrote it, because anyone can change the data and compute a new hash. A digital signature uses a private key, so it also proves who wrote the data.

In ASUR. ASUR uses hashing only. It has no keys by design, so it protects against accidents and corruption, not against a person who has access to the folder.

> **Q. What is the difference between a hash, a MAC and a digital signature?**
> **A.** A hash needs no key and gives integrity only. A MAC (such as HMAC) uses one shared secret key and proves the sender knew that key. A digital signature uses a private key to sign and a public key to verify, so anyone can verify but only the owner can sign.

2.5 Content addressing

Simple meaning. Content addressing means the name of a thing is made from its content. Same content gives the same name. Changed content gives a new name. Git commits and Docker image layers work this way.

In ASUR. An artifact id is the kind plus the first 12 characters of the body hash, for example `script-3fa9c1d2e4b7`. This is done in `asur/core/envelope.py`.

> **Q. What do you gain from content addressing?**
> **A.** Three things. Duplicates are easy to find, because same content has the same name. Changes are easy to detect, because the name changes. And caching is safe, because a name can never point to different content later.

2.6 Provenance and lineage

Simple meaning. Provenance is the record of where something came from. Lineage is the chain of inputs that led to an output.

In ASUR. Each artifact has a `sources` list with the id, version and checksum of every artifact it was built from. Each media file also records the model, the prompt, the seed and the licence that made it.

> **Q. Why record lineage?**
> **A.** For debugging and for trust. When an output is wrong, you can walk back through its inputs to find the step that caused it. And you can prove which exact inputs produced the final video.

3. Safe storage on disk

Asked in. The fundamentals round, and storage questions in system design.

3.1 Append-only storage and versioning

Simple meaning. Append-only means you only add new data. You never edit or delete old data. Each change creates a new version beside the old one.

In ASUR. Each save creates a new file such as `v0002-script-....json` next to `v0001`. The old file is never touched.

```
<project>/.script/
  index.json                          latest version of each kind
  artifacts/<kind>/v0001-<artifact_id>.json
  artifacts/<kind>/v0002-<artifact_id>.json
```

> **Q. What are the benefits and costs of append-only storage?**
> **A.** Benefits: you have full history, you can go back to any version, and there are no lost updates because nobody overwrites. Costs: it uses more disk, and you need an index to find the latest version quickly.

3.2 Race condition

Simple meaning. A race condition happens when two programs do the same thing at the same time and the result depends on who is faster. A common case is 'check, then act': both programs check that a file does not exist, then both write it, and one write is lost.

In ASUR. Two runs could try to save the same version at the same time. ASUR prevents this with the exclusive create flag explained next.

> **Q. What is a TOCTOU bug?**
> **A.** Time of check to time of use. You check a condition, and before you act on it, something else changes it. The fix is to make the check and the action one single step that cannot be split.

3.3 Exclusive create (O_EXCL)

Simple meaning. O_EXCL is a flag for opening a file. It tells the operating system: create this file only if it does not exist, otherwise fail. The check and the create happen as one step, so two programs can never both succeed.

In ASUR. Every artifact file is created with `O_WRONLY | O_CREAT | O_EXCL`. If the file is already there, the save fails instead of overwriting.

```
fd = os.open(path, os.O_WRONLY | os.O_CREAT | os.O_EXCL)   # fails if path exists
```

Try it. Run this yourself.

```
import os
flags = os.O_WRONLY | os.O_CREAT | os.O_EXCL
os.close(os.open("demo.txt", flags))    # works: the file is created
os.open("demo.txt", flags)              # FileExistsError: it will not overwrite
```

> **Q. How do you make sure two processes never write the same file?**
> **A.** Open the file with O_CREAT and O_EXCL. The operating system makes this one atomic step, so exactly one process creates the file and the other gets an error.

3.4 Atomic write (write then rename)

Simple meaning. Atomic means all or nothing. If a program crashes in the middle of writing a file, the file is left half-written. The safe way is to write a temporary file first, then rename it over the real file. The rename is one step, so a reader sees either the full old file or the full new file.

In ASUR. The index file `index.json` is the only file that changes. It is written to a temporary file in the same folder and then swapped in with `os.replace`.

Try it. Run this yourself.

```
import json, os, tempfile

def atomic_write(path, data):
    fd, tmp = tempfile.mkstemp(dir=os.path.dirname(path) or ".")
    with os.fdopen(fd, "w") as f:
        json.dump(data, f)
    os.replace(tmp, path)       # one step: readers see old or new, never half

atomic_write("index.json", {"script": 2})
```

> **Q. Why must the temporary file be in the same folder?**
> **A.** A rename is atomic only inside one file system. If the temporary file is on another disk, the rename becomes a copy and a delete, which is not atomic.

3.5 Lock file

Simple meaning. A lock makes sure only one worker does a job at a time. A lock file is a simple lock: whoever creates the file first owns the job.

In ASUR. Before the checker evaluates a video, it creates `<task_id>.claim` with O_EXCL. A second checker gets an error. This is in `asur/tools/queue.py`.

> **Q. What is the risk of a lock file?**
> **A.** If the owner crashes, the lock file stays and the job is blocked forever. This is called a stale lock. The usual fixes are a timeout, or storing the owner and time inside the lock so it can be released safely.

3.6 Idempotency

Simple meaning. An operation is idempotent if doing it twice gives the same result as doing it once. It makes retries safe.

In ASUR. Sealing the same bundle twice with the same bytes is allowed and changes nothing. Sealing it again with different bytes raises an error.

> **Q. Why is idempotency important in real systems?**
> **A.** Networks and programs fail, so work gets retried. If an operation is not idempotent, a retry can charge a customer twice or create a duplicate record. Idempotent operations can be retried without fear.

3.7 JSON Lines log

Simple meaning. JSON Lines is a file where each line is one small JSON object. You add a new line for each event. It is simple to append and simple to read line by line.

In ASUR. The run log (`.asur/runs/<run_id>.jsonl`), the hand-off queue and the hook memory (`.script/memory/hooks.jsonl`) all use it.

Try it. Run this yourself.

```
import json
with open("log.jsonl", "a") as f:                    # "a" = append
    f.write(json.dumps({"event": "saved"}) + "
")
events = [json.loads(line) for line in open("log.jsonl")]
print(events)
```

> **Q. Why JSON Lines and not one big JSON file?**
> **A.** To add one record to a big JSON file you must rewrite the whole file. With JSON Lines you only append one line. A crash can damage only the last line, not the whole file.

4. State machine and gates

Asked in. Low-level design, machine coding and system design rounds.

4.1 Finite state machine

Simple meaning. A state machine is a model where a thing is always in exactly one state, and it can move only along allowed paths. It makes long processes easy to follow, because you always know where you are and what can happen next.

In ASUR. A project is always in one of 18 states. The only allowed move is one step forward. The list is in `asur/orchestrator/states.py`, and the allowed moves are built from the order of the list, so a skip cannot be written by mistake.

#	State	Stage	Cost
0	DRAFT	0	cheap
1	INPUTS_VERIFIED	1	cheap
2	RESEARCH_COMPLETE	4	cheap
3	STRATEGY_APPROVED	5	cheap
4	HOOK_APPROVED	6	cheap
5	SCRIPT_APPROVED	7	moderate
6	TIMING_VERIFIED	9	moderate
7	CREATIVE_DIRECTION_APPROVED	11	moderate
8	SCREENPLAY_APPROVED	12	moderate

#	State	Stage	Cost
9	HERO_PROOF_READY	16	moderate
10	**HERO_PROOF_APPROVED** (a human must approve)	18	moderate
11	FULL_GENERATION	19	expensive
12	QA	20	expensive
13	VIRAL_CHECK_APPROVED	23	expensive
14	**PUBLISH_READY** (a human must approve)	25	expensive
15	PUBLISHED	25	expensive
16	PERFORMANCE_CAPTURED	26	expensive
17	LEARNED (end)	26	expensive

Try it. A forward-only state machine in ten lines.

```
STATES = ["DRAFT", "SCRIPTED", "RENDERED", "PUBLISHED"]
NEXT = dict(zip(STATES, STATES[1:]))      # each state -> the one after it

def advance(state):
    if state not in NEXT:
        raise ValueError("no move from " + state)
    return NEXT[state]

print(advance("DRAFT"))         # SCRIPTED
print(advance("PUBLISHED"))     # ValueError: it is the last state
```

Q. Why use a state machine instead of plain if/else code?
A. With many steps, if/else code hides the current position inside many flags. A state machine has one clear state and one table of allowed moves. Illegal moves are impossible, and the process is easy to test and to log.

Q. Where are state machines used in the real world?
A. Order status in e-commerce, payment status, TCP connections, CI pipelines, and workflow tools such as AWS Step Functions and Airflow.

4.2 Gate

Simple meaning. A gate is a checkpoint between two steps. It runs some checks and decides if the work may move forward.

In ASUR. A gate in `asur/core/gate.py` returns one of three answers. It runs every check and keeps the worst answer.

Answer	Meaning
SHIP	Go ahead. Every check passed.
HOLD	Wait. Something is missing, but nothing is wrong.
BLOCK	Stop. Something is wrong.

4.3 Fail closed and fail open

Simple meaning. When a check itself breaks, the system must choose. Fail open means 'allow when unsure'. Fail closed means 'deny when unsure'. Safety systems should fail closed.

In ASUR. If no checks are given, the gate returns HOLD. If a check crashes, the gate returns BLOCK. If a check returns an unknown word, the gate returns BLOCK. So a broken check can never pass by mistake.

Try it. A fail-closed gate. This is the same logic as asur/core/gate.py.

```
RANK = {"SHIP": 0, "HOLD": 1, "BLOCK": 2}

def run_gate(checks):
    if not checks:
        return "HOLD"                  # nothing was checked
    worst = "SHIP"
    for check in checks:
        try:
            result = check()
        except Exception:
            result = "BLOCK"           # a broken check never passes
        if result not in RANK:
            result = "BLOCK"           # an unknown answer never passes
        worst = max(worst, result, key=RANK.get)
    return worst

print(run_gate([]))                                # HOLD
print(run_gate([lambda: "SHIP", lambda: 1 / 0]))   # BLOCK
```

> **Q. Give one example each of fail open and fail closed.**
> **A.** Fail open: a door lock that opens when power is lost, so people can get out. Fail closed: a login system that rejects everyone when the password database is down. You choose based on which mistake is more costly.

4.4 Human in the loop

Simple meaning. Human in the loop means a person must approve before the system does something costly or hard to undo. A related idea is separation of duties: the person who makes the work should not be the one who approves it.

In ASUR. Two states need a person: the small sample before full generation, and the final publish. If nobody approves, the result is HOLD. If the maker approves their own work, it is recorded honestly as `SELF_APPROVED`. The check uses the local user name and git email from `asur/core/identity.py`.

> **Q. What is the four-eyes principle?**
> **A.** An important action needs two different people: one to do it and one to approve it. Code review is the common example. It reduces mistakes and misuse.

4.5 Orchestrator and control tokens

Simple meaning. An orchestrator is one central part that tells the other parts when to run. It does not do the work. It only tracks progress.

In ASUR. The orchestrator in `asur/orchestrator/orchestrator.py` carries only a small control token: state, gate, answer, digest and a retry count. It never carries the content itself.

```
step():
  1. find the next state
  2. if the next state is costly and the sample is not approved -> BLOCK
  3. run the gate for that state
  4. write one line to the run log
  5. SHIP -> move to the next state; otherwise stay and count the retry
```

Q. What is the difference between orchestration and choreography?
A. In orchestration, one central controller tells each part what to do. In choreography there is no controller: each part reacts to events from the others. Orchestration is easier to follow and debug. Choreography is looser and scales well, but the full flow is harder to see.

4.6 Allow-list and deny-list

Simple meaning. An allow-list names what is permitted and rejects everything else. A deny-list names what is banned and permits everything else. An allow-list is safer, because a new unknown item is rejected by default.

In ASUR. The run log accepts only seven keys: state, next_state, gate, disposition, substate, digest, residency. Any other key raises an error, so private content cannot slip into the log.

Q. How do you stop secrets from leaking into logs?
A. Use an allow-list at the logger: only named safe fields can be written. Do not log whole objects. This makes a leak impossible instead of just discouraged.

4.7 Cheap work first

Simple meaning. If a mistake will be found anyway, find it at the cheapest step. Run fast, free checks before slow, costly work.

In ASUR. Each stage has a cost level. The costly stages (full generation and render) are blocked until a human has approved a small sample.

Q. What does 'shift left' mean?
A. Move checks earlier in the process. A bug found while writing code costs minutes. The same bug found in production costs hours and reputation.

5. Architecture and design patterns

Asked in. Low-level design and Python rounds.

5.1 Layers and the dependency rule

Simple meaning. In a layered design, code is split into groups, and each group may depend only on certain other groups. The stable central code should not depend on heavy or changing outer code.

In ASUR. The control code (core, script, viral check, orchestrator) imports only the Python standard library. Heavy media libraries are used only inside the adapter files. So the core runs with zero installed packages.

Q. How do you keep an architecture rule from being broken over time?
A. Turn the rule into an automated test. ASUR has a test that reads every control file and fails if it imports anything outside the standard library. A rule that lives only in a document gets broken.

5.2 Adapter pattern

Simple meaning. An adapter is a thin wrapper that makes an outside tool fit the shape your code expects. Your code talks to the adapter, and the adapter talks to the tool. To change the tool, you change only the adapter.

In ASUR. Each model has one file in `asur/generation/adapters/`: FFmpeg, SDXL, Kokoro, Stable Audio, faster-whisper. Each takes a job and returns a result with provenance. The planning code never imports a model library.

> **Q. How would you replace the image model with a different one?**
> **A.** Write one new adapter with the same input and output shape, and add one line to the routing table. The planning code and its tests do not change.

5.3 Registry (dispatch table)

Simple meaning. A registry is a lookup table from a name to the code that handles it. It replaces a long if/elif chain. Adding a new case means adding one row.

In ASUR. `_ROUTES` in `asur/generation/dispatch.py` maps a model name to its adapter module and function.

```
_ROUTES = {
    "FFmpeg":     ("asur.generation.adapters.ffmpeg", "render"),
    "SDXL":       ("asur.generation.adapters.sdxl", "generate"),
    "Kokoro-82M": ("asur.generation.adapters.kokoro", "synthesize"),
}
module_path, fn_name = _ROUTES[model]
fn = getattr(importlib.import_module(module_path), fn_name)
```

> **Q. What is the open/closed principle?**
> **A.** Code should be open for extension and closed for modification. You can add new behaviour without editing existing, tested code. A registry does this: a new model is a new row, not a change to old logic.

5.4 Lazy import and optional dependencies

Simple meaning. A lazy import loads a library only at the moment it is needed, inside the function, not at the top of the file. An optional dependency is a library that is installed only by users who want that feature.

In ASUR. The adapters import `torch` and `diffusers` inside the function body. The heavy libraries are in an optional install, `asur[generation]`. The route table stores module names as text, so nothing heavy loads until a job runs.

Try it. The pattern.

```
def make_image(prompt):
    import torch            # loaded only when this function is called
    ...

# importing this file is instant, even if torch is not installed
```

> **Q. When is a lazy import useful?**
> **A.** When a library is slow to load or may not be installed. The program starts fast, and users who do not use that feature never need the library.

5.5 Dependency injection

Simple meaning. Dependency injection means a function receives the things it needs as arguments, instead of creating or finding them itself. This makes the function easy to test, because a test can pass in a fake.

In ASUR. Every builder accepts an `identity=` argument. In real use it comes from the machine. In tests, a fixed identity is passed in, so tests do not depend on the machine's git settings.

Try it. The pattern.

```
def build(idea, identity=None):
    identity = identity or get_identity()      # real machine value by default
    ...

build("my idea", identity={"git_email": "test@example.com"})    # in a test
```

> **Q. Why does dependency injection help testing?**
> **A.** The test controls every input. It can pass a fake clock, a fake user or a temporary folder, so the result is the same on every machine.

5.6 Configuration as data

Simple meaning. Instead of writing platform differences as if/else in many places, put them in one data object and pass that object around.

In ASUR. A platform is one frozen dataclass, `PlatformPack`, in `asur/platforms/`. It holds size, aspect ratio and duration. Frozen means it cannot be changed after it is created.

Platform	Aspect	Size	Default length
instagram_reels (default)	9:16	1080 x 1920	60 s
youtube_shorts	9:16	1080 x 1920	45 s
youtube_long	16:9	1920 x 1080	480 s

Try it. Run this yourself.

```
from dataclasses import dataclass

@dataclass(frozen=True)
class Format:
    width: int
    height: int

f = Format(1080, 1920)
f.width = 720               # FrozenInstanceError: it cannot be changed
```

> **Q. What is an immutable object and why use one?**
> **A.** It is an object that cannot change after creation. It is safe to share between many parts of the program, because no part can change it and surprise the others.

5.7 Backward compatibility

Simple meaning. A change is backward compatible when existing users see no difference after it.

In ASUR. The new `platform=` argument is optional. Tests check that leaving it out gives output that is byte-for-byte the same as before.

> **Q. How do you add a feature without breaking existing users?**
> **A.** Make the new option optional with a default that keeps the old behaviour. Then write a test that proves the old output is unchanged.

5.8 Pipeline pattern

Simple meaning. A pipeline is a chain of steps where each step takes the output of the step before it. Each step does one job.

In ASUR. Idea -> research -> audience -> strategy -> hooks -> script. Each stage reads the latest saved artifact and writes its own.

> **Q. What is the benefit of small pipeline stages?**
> **A.** Each stage can be tested alone, re-run alone, and replaced alone. When something fails, you know exactly which stage failed.

6. The firewall: keeping maker and checker apart

Asked in. System design rounds, and evaluation questions for AI roles.

6.1 Goodhart's law

Simple meaning. When a measure becomes a target, it stops being a good measure. If the maker knows exactly how it will be scored, it learns to please the scorer instead of doing good work. Students who study only the exam questions are an example.

In ASUR. GENERATION makes the video. VIRAL CHECK scores it. If the maker could see the scoring rules, the score would become meaningless. So the two never share data or code.

> **Q. Why should the tester be separate from the builder?**
> **A.** A builder who knows the test can pass the test without solving the real problem. In machine learning this is called training on the test set. Keeping them apart keeps the result honest.

6.2 Projection (filtered view)

Simple meaning. A projection is a filtered copy of data that contains only the fields one reader is allowed to see. This follows the principle of least privilege: give each part only what it needs.

In ASUR. SCRIPT builds two views in `asur/tools/projection.py`. Each view has a list of forbidden field names.

View	Carries	Forbidden names
GENERATION_VIEW	brief, hooks, quality_floors, cost_policy	exploit, finding_text, gate_thresholds, pass_criteria, verdict, viral_check, weakness
VIRAL_VIEW	bundle_hashes, provenance, platform_facts, rubric_metadata	hardness_targets, per_dimension_target, quality_floors, quality_targets, target_score, what_good_is

Before a view is returned, `conformance_scan()` walks every key and every text value. If a forbidden name appears, it raises `FirewallLeak`. A second guard is a test that fails if the two packages import each other.

> **Q. What is the principle of least privilege?**
> **A.** Every part of a system gets only the access it needs for its job, and nothing more. If that part has a bug or is misused, the damage stays small.

> **Q. What is the limit of a name-based filter?**
> **A.** It blocks forbidden names, not forbidden meaning. The same value under a harmless key would pass. So it stops accidents, and new fields still need a human review.

6.3 Digest binding

Simple meaning. Digest binding means an approval is tied to the exact bytes that were reviewed. If the content changes after review, the approval no longer matches.

In ASUR. The hand-off queue in `asur/tools/queue.py` has four steps. Release succeeds only when the digests agree.

Step	What happens
seal	Hash the bundle and store it. After this it is frozen.
claim	One checker takes the job, using a lock file.
verdict	The checker records the digest it looked at and its answer.
place	Release only if the sealed digest and the verdict digest are the same.

> **Q. How do you make sure the thing that was reviewed is the thing that ships?**
> **A.** Record the hash of the content inside the approval. At release time, hash the content again and compare. If someone changed it after review, the hashes differ and the release is refused.

6.4 Rule-based scoring

Simple meaning. A rule-based system gives a result from fixed, written rules, not from a trained model. Every result can be explained by pointing to the rule.

In ASUR. The viral check scores 12 separate dimensions and never joins them into one percentage. Each score has a written reason. It reads facts that are passed to it; it does not yet measure the video file itself, so say that plainly if asked.

> **Q. Why not give one 'viral score' out of 100?**
> **A.** Nobody can honestly predict virality with one number. One number hides which part is weak and gives false confidence. Separate scores with reasons tell the creator what to fix.

7. Determinism and learning

Asked in. Testing questions, and data or AI roles.

7.1 Deterministic code

Simple meaning. Deterministic means the same input always gives the same output. There is no hidden randomness and no dependence on time or machine.

In ASUR. All script stages are deterministic templates. The same idea always produces the same hooks and the same script.

> **Q. Why is determinism valuable?**
> **A.** Tests can check exact output. A bug can be reproduced by running the same input again. And two people get the same result on two machines.

7.2 Hash-seeded variation

Simple meaning. Sometimes you want variety without real randomness. You can take a hash of the input and use it as the 'random' number. It looks random but is the same on every run.

In ASUR. Each hook is scored on 15 dimensions. A small plus or minus is added to each score, taken from the SHA-256 of the hook text and the dimension name. This is in `asur/script/hooks.py`.

```
jitter = (sha256(text + "|" + dimension) scaled to 0..1  - 0.5) * 12     # about +/- 6
```

Try it. Run this yourself.

```
import hashlib

def jitter(text):
    n = int(hashlib.sha256(text.encode()).hexdigest()[:8], 16)
    return (n / 0xFFFFFFFF - 0.5) * 12

print(jitter("my hook|clarity"))    # looks random
print(jitter("my hook|clarity"))    # exactly the same number again
```

> **Q. What is a flaky test and how do you avoid one?**
> **A.** A flaky test passes sometimes and fails sometimes with no code change. Common causes are random values, time, network and test order. Avoid them by fixing the seed, passing the clock in, and using no network.

7.3 Evidence classes

Simple meaning. Not every claim is equally sure. Labelling each claim with how strong its proof is stops a guess from being presented as a fact.

In ASUR. Every claim gets one of four labels through `evidence.claim()`.

Label	Meaning
VERIFIED_FACT	Publicly stated or directly seen.
STRONG_EVIDENCE	Supported by a credible study.
EMPIRICAL	Measured from this creator's own videos.
HYPOTHESIS	Believable, but not tested.

7.4 Feedback loop

Simple meaning. A feedback loop takes the real result of an action and feeds it back to improve the next action.

In ASUR. After publishing, the creator types the real numbers (views, completion, shares) into a file. `asur/learning/` compares them with what was expected and appends the result to the hook memory. The next run reads that memory, so a hook that was used too often is marked as tired.

> **Q. How many data points do you need before trusting a pattern?**
> **A.** One result proves nothing. You need many comparable cases, and you must compare with a baseline such as the creator's own average. With few samples you report it as a hint, not as a finding.

8. Video and audio concepts

Asked in. Video, streaming and media companies.

8.1 Container and codec

Simple meaning. A container is the file format that holds video and audio together, such as MP4. A codec is the method that compresses the video or audio inside it, such as H.264 for video and AAC for audio.

In ASUR. ASUR renders an MP4 file with H.264 video and the `yuv420p` pixel format, which plays on all phones.

Try it. Make a 3-second vertical clip, then ask ffprobe what is inside it.

```
ffmpeg -f lavfi -i color=c=navy:s=1080x1920:d=3 -r 30 -pix_fmt yuv420p card.mp4
ffprobe -v error -show_entries stream=codec_name,width,height,r_frame_rate         -of default=nw=1
card.mp4
# codec_name=h264  width=1080  height=1920  r_frame_rate=30/1
```

> **Q. What is the difference between MP4 and H.264?**
> **A.** MP4 is the box. H.264 is how the video inside the box is compressed. One MP4 file can hold H.264 or another codec, and the same H.264 video can sit inside MP4 or MKV.

8.2 Resolution, aspect ratio and frame rate

Simple meaning. Resolution is the size in pixels, such as 1080 x 1920. Aspect ratio is the shape, such as 9:16 for vertical video. Frame rate is how many pictures are shown per second, such as 30 fps.

In ASUR. These values come from the platform pack, so the same pipeline can produce a vertical reel or a wide YouTube video.

8.3 Bitrate and keyframes

Simple meaning. Bitrate is how much data is used per second of video. Higher bitrate means better quality and a bigger file. A keyframe is a full picture; the frames between keyframes store only what changed.

In ASUR. FFmpeg picks these during the render. Platforms re-compress every upload, so the render should be good quality to begin with.

> **Q. Why is video compressed with keyframes and changes?**
> **A.** Most of the picture stays the same from one frame to the next. Storing only the changes saves a lot of space. Keyframes are needed so that playback can start or jump to a point.

8.4 FFmpeg and the render

Simple meaning. FFmpeg is a free command-line tool that reads, changes and writes video and audio. A filter is one change, such as scale, crop or draw text. Concatenate means join clips end to end. Mux means put video and audio streams together in one file.

In ASUR. The adapter in `asur/generation/adapters/ffmpeg.py` makes one card per scene (an image scaled and cropped to fit, or a plain colour), burns the on-screen text with the `drawtext` filter, joins the cards, and muxes in the voice or music.

Try it. Run this yourself.

```
import subprocess
subprocess.run(["ffmpeg", "-y", "-i", "card.mp4", "-vf", "scale=540:960", "small.mp4"],
               check=True, timeout=300)
```

> **Q. How do you run an external program safely from Python?**
> **A.** Use `subprocess.run` with the arguments as a list, not as one shell string. Then user text is passed as plain data and cannot become a command. Also set a timeout and check the exit code.

8.5 Safe area

Simple meaning. Apps draw their own buttons and captions on top of a video. Text placed near the top or bottom edge gets hidden. The safe area is the middle part that stays visible.

In ASUR. `asur/editing/captions.py` keeps a 12% band free at the top and the bottom.

8.6 Decibels and ducking

Simple meaning. A decibel (dB) is the unit for loudness. It is a logarithmic scale: minus 6 dB is about half the amplitude. Ducking means lowering the music automatically while a person is speaking.

In ASUR. `asur/editing/audio_mix.py` sets default levels: voice at -3 dB and music at -18 dB, so the music never covers the voice.

8.7 Timeline

Simple meaning. A timeline is the data model of an edit: tracks (video, audio, text) that hold clips, and each clip has a start time and a length.

In ASUR. `asur/editing/timeline.py` has three dataclasses: `Clip`, `Track` and `Timeline`. Every edit creates a new version of the clip, so any change can be undone.

> **Q. What is a dataclass in Python?**
> **A.** It is a class made mainly to hold data. The `@dataclass` decorator writes the constructor, the comparison and the printing code for you. With `frozen=True` the object cannot be changed after creation.

9. AI model concepts

Asked in. Applied AI and ML engineer roles.

9.1 Diffusion model

Simple meaning. A diffusion model makes an image by starting from random noise and removing a little noise at each step, guided by the text prompt, until a clear picture appears.

In ASUR. The SDXL adapter makes the background images. SDXL was trained on images of about one megapixel, so ASUR generates at 832 x 1216 and scales up.

Setting	Meaning
Steps	How many times noise is removed. More steps: slower, usually cleaner.
Guidance scale	How strictly the image follows the prompt.
Seed	The number that fixes the starting noise. Same seed and prompt: same image.

Q. Why record the seed?
A. For reproducibility. With the same model, prompt, settings and seed, you get the same image again. Without the seed, you can never recreate an image you liked.

9.2 Text to speech and speech to text

Simple meaning. Text to speech (TTS) turns written text into a spoken voice. Speech to text (ASR) turns a voice recording into written text, with the time of each word.

In ASUR. Kokoro makes the voice-over. faster-whisper listens to the voice and returns word timings, which are used to show captions at the right moment.

Q. How are captions timed to the voice?
A. A speech-to-text model returns each word with a start and end time. The caption for a word is shown between those two times.

9.3 Inference and hardware

Simple meaning. Training teaches a model. Inference uses a trained model to produce output. Inference runs on a GPU when one is available: CUDA on NVIDIA cards, MPS on Apple Silicon, and CPU as the slow fallback.

In ASUR. ASUR only does inference. It never trains a model. The memory of the GPU (VRAM) decides which models a machine can run.

Q. What is the difference between training and inference?
A. Training changes the model's weights using a lot of data, and it is slow and costly. Inference keeps the weights fixed and just runs the model on new input.

9.4 Model licences

Simple meaning. A model that is free to download is not always free to use for business. Each model has a licence, and some allow research use only.

In ASUR. `asur/generation/license_guard.py` keeps a list of models with risky licences. It checks the plan before any generation starts and names the asset that is the problem.

> **Q. Why check licences in code and not by hand?**
> **A.** People forget. A check in code runs every time, and it runs before any costly work is done.

9.5 Tests for code that needs a GPU

Simple meaning. Some code cannot run on a normal test server because it needs a GPU or a large download.

In ASUR. The real model calls are marked `# pragma: no cover` and are run by a person on a capable machine. Everything around them (planning, routing, error handling) is tested normally.

10. Testing and code quality

Asked in. Every backend interview; more deeply for SDET and DevOps roles.

10.1 Unit and integration tests

Simple meaning. A unit test checks one small piece alone, and is fast. An integration test checks that several pieces work together, often with a real outside tool, and is slower.

In ASUR. The unit tests use no network, no GPU and no FFmpeg. The integration tests in `tests/integration/` call the real FFmpeg and skip themselves if it is not installed.

Try it. Run this from the ASUR folder.

```
python3 -m pytest tests/test_core.py -q        # one file of unit tests
python3 -m pytest tests -q                     # all unit tests
python3 -m pytest tests -q -m integration      # the tests that need ffmpeg
```

> **Q. What is the test pyramid?**
> **A.** Many fast unit tests at the bottom, fewer integration tests in the middle, and very few slow end-to-end tests at the top. This gives quick feedback and still covers the full system.

10.2 Test isolation

Simple meaning. Each test should set up what it needs and leave nothing behind. A test must not depend on another test, on the network, or on the machine it runs on.

In ASUR. Tests write into temporary folders and pass in a fixed identity. No test touches the network.

10.3 Negative tests

Simple meaning. A positive test checks that correct input works. A negative test checks that wrong input is rejected in the right way.

In ASUR. Every fail-closed path has a test that feeds it bad input and expects the exact error: a wrong checksum, a missing approver, a leaking view.

> **Q. How do you know a safety check really works?**
> **A.** Break the rule on purpose and confirm the check fails. A check that has never failed may not be checking anything.

10.4 Architecture tests

Simple meaning. An architecture test checks the structure of the code, not its behaviour.

In ASUR. `tests/test_boundary.py` uses Python's `ast` module to read the imports of every control file and fails on a forbidden one. `tests/test_firewall.py` fails if the maker and the checker import each other.

Try it. Print every import in a file without running the file.

```
import ast
tree = ast.parse(open("asur/core/gate.py").read())
for node in ast.walk(tree):
    if isinstance(node, ast.Import):
        print([n.name for n in node.names])
    elif isinstance(node, ast.ImportFrom):
        print(node.module)
```

> **Q. What is an AST?**
> **A.** Abstract syntax tree. It is the code turned into a tree of nodes, such as imports, functions and calls. Tools read the tree to inspect code without running it. Linters work this way.

10.5 Linting and type checking

Simple meaning. A linter reads code and reports style problems and likely bugs. A type checker verifies that values are used with the right types, without running the code.

In ASUR. ASUR uses `ruff` as the linter and `mypy` as the type checker.

10.6 Continuous integration

Simple meaning. Continuous integration (CI) runs the checks automatically on every push, so a problem is found within minutes.

In ASUR. The workflow in `.github/workflows/ci.yml` runs ruff, mypy and the tests. It also installs the package with no extras and confirms the core still imports.

> **Q. What is the difference between CI and CD?**
> **A.** CI builds and tests every change automatically. CD goes one step further and delivers the tested change to users automatically.

10.7 Architecture decision record

Simple meaning. An ADR is a short note that records one design decision: the situation, the choice and its results. It answers 'why is it built this way?' for future engineers.

In ASUR. ASUR keeps its ADRs in `docs/adr/`.

11. Security concepts

Asked in. Security questions in backend rounds, and security roles.

11.1 Trust boundary and threat model

Simple meaning. A trust boundary is the line between what you trust and what you do not. A threat model is a written list of what can go wrong and what you do about it.

In ASUR. ASUR trusts the local user and the local disk. It does not trust downloaded model files, third-party packages or input media files.

> **Q. How do you do threat modelling?**
> **A.** Answer four questions. What are we building? What can go wrong? What will we do about it? Did we do enough? It is also important to write down what you are not defending against.

11.2 Supply-chain risk

Simple meaning. Every package you install is code written by someone else that runs with your permissions. If one package is hacked, your program is hacked.

In ASUR. The core has zero third-party packages, so this risk is zero there. The risk exists only in the optional generation install.

> **Q. How do you reduce supply-chain risk?**
> **A.** Use fewer dependencies. Pin exact versions. Scan them for known problems. And keep risky dependencies away from the sensitive part of the system.

11.3 Command injection

Simple meaning. Injection happens when user text is treated as part of a command. If text is pasted into a shell command, special characters can run something else.

In ASUR. On-screen text goes into an FFmpeg filter. The adapter escapes special characters in `_escape_drawtext` and passes arguments as a list, never through a shell.

Try it. Run this yourself.

```
import subprocess
text = "hello; ls"
subprocess.run("echo " + text, shell=True)    # unsafe: also runs ls
subprocess.run(["echo", text])                # safe: prints the text as it is
```

> **Q. What is the general defence against injection?**
> **A.** Keep code and data apart. Pass user input as a parameter, not by building a string. For SQL that means parameterised queries. For processes it means an argument list with no shell.

11.4 Unsafe file formats

Simple meaning. Some file formats can run code when they are opened. Python's pickle is one: loading a pickle file from a stranger can execute their code.

In ASUR. Old model files use pickle. The safer format is `safetensors`, which holds only numbers and cannot run code.

> **Q. Why is pickle dangerous?**
> **A.** Loading a pickle rebuilds objects by calling code named inside the file. An attacker can name any function, so loading the file runs their code.

11.5 Identity versus authentication

Simple meaning. Identity is a name: who you say you are. Authentication is proof of that name, such as a password or a key.

In ASUR. ASUR records identity only: the OS user and the git name and email. It has no authentication by design. This gives an honest record of who ran what, but it is not access control.

> **Q. What is the difference between authentication and authorisation?**
> **A.** Authentication checks who you are. Authorisation checks what you are allowed to do. Login is authentication; a role or permission check is authorisation.

12. Python features used

Asked in. The Python round.

Module	Used for
`hashlib`	Makes SHA-256 hashes.
`json`	Reads and writes JSON; `sort_keys` gives the canonical form.
`pathlib`	Works with file paths as objects, not plain text.
`os.open`, `os.replace`	Exclusive create and atomic rename.
`subprocess`	Runs FFmpeg as a separate program.
`argparse`	Builds the command line: `asur script`, `asur run`, `asur render`.
`dataclasses`	Small data-holding classes; `frozen=True` makes them fixed.
`importlib`	Imports a module by its name as text, used for lazy loading.
`ast`	Reads source code as a tree, used by the architecture test.
`getpass`	Gets the current OS user name.
`pytest`	The test runner. Markers separate unit and integration tests.

> **Q. What is the difference between a module and a package in Python?**
> **A.** A module is one `.py` file. A package is a folder of modules. `asur` is a package; `asur/core/gate.py` is a module inside it.

> **Q. What does 'standard library only' mean and why does it matter?**
> **A.** The code imports only what ships with Python. Nothing has to be installed, nothing can break from a package update, and there is no third-party code to audit.

13. One-line revision

Term	One line
Hash	A fixed-size fingerprint of data.
Canonical form	One fixed way to write data so equal data gives equal bytes.
Checksum	A stored hash used to detect change.
Signature	A hash made with a private key; proves who wrote it.
Content addressing	Naming a thing by the hash of its content.
Lineage	The chain of inputs behind an output.
Append-only	Add new versions; never edit old ones.
Race condition	The result depends on timing between two actors.
Atomic	All or nothing; no half-done state is visible.
Idempotent	Doing it twice equals doing it once.
State machine	One current state and a table of allowed moves.
Gate	A checkpoint that answers SHIP, HOLD or BLOCK.
Fail closed	When unsure, deny.
Allow-list	Name what is permitted; reject the rest.
Orchestrator	A central controller that tracks progress.
Adapter	A wrapper that makes an outside tool fit your code.
Registry	A table from a name to its handler.
Lazy import	Load a library only when it is needed.
Dependency injection	Pass in what a function needs.
Immutable	Cannot change after creation.
Least privilege	Give each part only the access it needs.
Goodhart's law	A measure that becomes a target stops measuring.
Digest binding	An approval tied to the exact bytes reviewed.
Deterministic	Same input, same output.
Container / codec	The file box / the compression inside it.
Mux	Join video and audio streams into one file.
Diffusion	Make an image by removing noise step by step.
Seed	The number that makes a random result repeatable.
Inference	Using a trained model to produce output.
Unit / integration test	One piece alone / pieces working together.
AST	Source code as a tree.

Term	One line
CI	Automatic checks on every change.
Trust boundary	The line between trusted and untrusted.
Injection	User text treated as a command.

14. Practice tasks and follow-up questions

Asked in. Machine coding rounds (the tasks) and the project round (the follow-ups).

14.1 Coding tasks

Write each one from memory, with a small test. Then compare with the ASUR file.

#	Task	Time	See
1	A function that gives the same hash for equal dictionaries.	10 min	2.2
2	An atomic file write.	10 min	3.4
3	A lock that only one process can take.	10 min	3.3, 3.5
4	A version store: each save makes v1, v2, v3 and never overwrites.	30 min	3.1
5	A forward-only state machine that logs every move.	20 min	4.1
6	A gate that runs checks and fails closed.	15 min	4.3
7	A logger that rejects any key outside an allow-list.	15 min	4.6
8	A plugin registry that loads a handler by name.	15 min	5.3
9	A test that fails if a file imports a banned module.	20 min	10.4
10	Join three clips and add an audio track with FFmpeg.	20 min	8.4

14.2 Follow-up questions about your project

Q. What happens if the power goes off in the middle of a save?
A. The artifact file can be left half-written. On the next read its checksum does not match, so the bad data is rejected and never used. The index file is safe, because it is written to a temporary file and renamed. To improve it, I would write artifacts through a temporary file too.

Q. Two runs save the same version at the same moment. What happens?
A. Both try to create the same file name with O_EXCL. The operating system lets exactly one succeed. The other gets an error instead of overwriting, so no data is lost.

Q. A render takes ten minutes and the program is closed. What happens?
A. The saved artifacts stay on disk, but the orchestrator keeps its current state in memory, so the run starts again from the first state. The fix is to rebuild the state at start-up by replaying the run log.

Q. Why did you use files and not a database?
A. It is a single-user tool on one machine. Files are simple, easy to inspect, and need no server. I would move to a database when there are many users, many writers at the same time, or a need to search across projects.

Q. How would you turn this into a service for 1,000 users?
A. Put an API server in front. Each request becomes a job in a queue. Worker machines with GPUs take jobs from the queue. Media files go to object storage, named by their hash. Project state and versions go to a database. Add login, and send progress to the browser. The state machine and the gates stay the same.

Q. What is the weakest part of your project?
A. The viral check. It scores facts that are passed to it; it does not measure the video file. I would first read real facts from the file with ffprobe, then compare the scores with real results from published videos to see if the scores mean anything.

Q. How do you know your safety checks work?
A. Each one has a negative test that breaks the rule on purpose and expects the exact error. A check that has never been seen to fail may not be checking anything.